

SIMATS ENGINEERING

Co3-ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks**COURSE CODE:**CSA07

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

QUESTIONS:4 Total Marks:40

Code of Conduct:

HARISH KUMAR M (192565053) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

HARISH KUMAR.M

Criteria	Understanding the case 2.5	Application of Concepts	Analysis 2	Solution Design	Clarity 1	Total (10)
----------	----------------------------	--------------------------------	----------------------	------------------------	---------------------	----------------------

2.5	2
Q1	
Q2	
Q3	
Q4	

Faculty Signature

QUESTIONS:4 Total Marks:40

1. A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss, 180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

- 1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.

The symptoms described — voice distortion, frozen video frames, delayed communication, 8% packet loss, 180 ms jitter, and fluctuating bandwidth — point to a transport-layer-driven problem that is amplified by application-layer buffering decisions. In other words, both layers are implicated, but the *root cause* sits at the transport layer, while the *severity of the user-perceived symptoms* is determined by how the application layer handles that transport behavior.

Transport layer angle:Most video conferencing platforms use UDP (often via RTP/RTCP, or wrapped in proprietary protocols like SRTP over UDP, or QUIC) rather than TCP for real-time audio/video, precisely because TCP's reliability mechanisms are unsuitable for live media. If this platform (or a portion of its media path —

e.g., signaling, screen-share, or a fallback path) is using TCP, the described symptoms are classic evidence of it:

- TCP's retransmission mechanism means lost packets are resent rather than skipped — but by the time a retransmitted audio/video packet arrives, it's often already too late to be useful, causing stutter and frozen frames rather than smooth degradation.
- TCP's congestion control (slow start, AIMD back-off) reduces throughput sharply upon detecting loss, which explains the fluctuating bandwidth utilization.
- TCP's in-order delivery guarantee means a single lost/delayed packet head-of-line blocks all subsequent packets, which directly explains delayed communication and jitter buildup.

If UDP is already in use, then the 8% packet loss and 180 ms jitter are more likely network-path issues (congestion, poor QoS marking, Wi-Fi contention) — but how badly this affects the user still depends heavily on...

Application layer angle:

- If the jitter buffer is too small, natural network jitter (180 ms is quite high) causes late packets to be discarded, producing choppy audio and frozen video.
- If the jitter buffer is too large (a common "safe" over-correction), it introduces excessive end-to-end delay, causing the perceived lag/delayed communication even without much real loss.
- Poor adaptive bitrate/congestion response at the application layer means the app doesn't scale video quality down fast enough when bandwidth fluctuates, causing frame freezes rather than graceful quality reduction.
- Lack of forward error correction (FEC) or packet loss concealment (PLC) at the app layer means every lost packet directly translates to audible/visible artifacts instead of being masked.

1. Justify your answer based on the communication requirements of real-time multimedia applications.

Real-time multimedia (voice/video conferencing) has fundamentally different requirements than typical data transfer:

Requirement	Real-time media need	TCP behavior	UDP behavior
Timeliness	Packets must arrive within tens of milliseconds or become useless	Retransmits late packets anyway → wasted effort, added delay	Drops late/lost packets, no retransmission → keeps stream moving
Ordering	Minor reordering tolerable (jitter buffer handles it)	Strict in-order delivery → head-of-line blocking	Delivers as received, app reorders if needed
Loss tolerance	Small losses tolerable via	Zero loss tolerance	Loss passed to app,

	codec concealment/FEC; retransmission not useful	by design — retransmits everything	which can conceal it
Throughput stability	Needs adaptive, graceful bitrate scaling	Congestion control causes sawtooth throughput drops	App/codec must manage adaptation itself
Latency budget	ITU-T G.114 recommends <150 ms one-way for acceptable voice quality; jitter should ideally be <30 ms	Adds unpredictable retransmission delay	Minimal transport- added delay

Given this, an 8% packet loss is within a range that well-designed UDP+RTP systems with FEC/PLC can conceal reasonably well, but it will devastate a TCP-based or poorly-buffered stream. Similarly, 180 ms jitter is **6x higher** than the ideal threshold — this alone, combined with poor buffer tuning, explains the frozen frames and choppy audio independent of the raw loss rate.

This confirms real-time multimedia is fundamentally a "**deliver on time, tolerate some loss**" workload — the opposite of what TCP optimizes for — while the application layer must actively compensate for whatever the network/transport layer cannot guarantee.

1. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

Protocol-level (Transport):

- Ensure media streams use **UDP-based RTP/RTCP** (or **QUIC**, which gives some reliability benefits without TCP's head-of-line blocking) rather than TCP for audio/video.
- Reserve TCP only for signaling (SIP/WebSocket) and non-real-time data (file transfer, chat).
- Implement/verify **QoS marking (DSCP/DiffServ, e.g., EF for voice)** on the network so conferencing traffic is prioritized over best-effort traffic during congestion.
- Use **WebRTC's built-in congestion control (e.g., Google Congestion Control/BBR-based)** if not already active, to adapt sending rate to real-time network conditions instead of relying on TCP-style AIMD.

Application-level:

- Tune the **adaptive jitter buffer** dynamically (not fixed-size) so it expands/contracts based on measured jitter, balancing latency vs.

smoothness rather than using a static buffer.

- Enable **Forward Error Correction (FEC)** and **Packet Loss Concealment (PLC)** in the codec (e.g., Opus has built-in FEC and PLC) to mask losses up to ~10% without visible/audible artifacts.
- Implement **adaptive bitrate/resolution scaling** so video quality steps down smoothly under bandwidth pressure instead of freezing frames.
- Use **NACK (negative acknowledgment) + selective retransmission** for key frames only (not full reliable delivery) — RTP supports this via RTCP feedback, giving some reliability without TCP's blocking behavior.
- Deploy **simulcast/SVC (Scalable Video Coding)** so the server can drop to lower layers per-recipient based on their individual network conditions, rather than one-size-fits-all quality.

Network/infrastructure-level (supporting both layers):

- Audit peak-hour bandwidth provisioning and Wi-Fi/LAN congestion at the client side, since fluctuating bandwidth utilization during "peak business hours" suggests contention, not just protocol inefficiency.
- Consider **edge/regional media relay servers (SFU architecture)** to reduce path length and jitter accumulation for multinational offices.

2. A multinational software company hosts its web services across multiple continents.

Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

1. Analyze the reasons behind the high DNS resolution time.

An 800 ms average DNS lookup is roughly **8–16x higher** than acceptable (well-optimized DNS resolution should be **20–50 ms**). For a multinational company, likely causes include:

a) Geographic distance to authoritative servers

- If the company hosts authoritative DNS servers in only one or two regions (e.g., only in the US), employees in Asia-Pacific or Europe must send queries across continents, adding **100–300 ms of pure propagation delay per round trip**, especially with recursive resolution requiring multiple hops.

b) Deep recursive resolution chain

- A full recursive lookup (Root → TLD → Authoritative) for an uncached domain can require **3+ sequential round trips** before the browser gets an answer. If intermediate resolvers aren't caching effectively, every lookup pays this full cost.

c) Low or misconfigured TTL (Time-To-Live) values

- If TTL is set very low (e.g., a few seconds), DNS caches at the resolver, ISP, and

OS/browser level expire quickly, forcing **frequent re-resolution** instead of serving from cache — a very common cause of consistently high average lookup time.

d) Single points of failure / lack of geographic redundancy

- With a single (or few) authoritative DNS server locations using traditional Unicast addressing, all global queries converge on the same physical servers, creating a **latency and load bottleneck** during peak hours, and worsening tail latency for far-away regions.

e) No use of Anycast routing

- Without Anycast, DNS queries are always routed to a fixed server IP regardless of the requester's location, rather than being automatically routed to the *nearest* available server via BGP.

f) Lack of client-side/application-level optimizations

- No DNS pre-fetching or pre-resolution means every new session pays the full DNS cost cold, rather than resolving domains proactively before they're needed.

g) DNSSEC or resolver misconfiguration overhead (secondary factor)

- If DNSSEC validation is enabled without efficient caching, or if resolvers are overloaded/under-provisioned, additional cryptographic validation and queuing delay can add tens to hundreds of milliseconds.

1. Propose a suitable DNS architecture using techniques such as **Anycast DNS**, **DNS pre-fetching**, and **TTL optimization** to reduce the lookup time below **50 ms**.

a) Anycast DNS deployment

- Deploy authoritative DNS servers at **multiple Points of Presence (PoPs)** across continents (North America, Europe, Asia, etc.), all advertising the **same Anycast IP address** via BGP.
- Network routing automatically directs each user's query to the **topologically nearest** DNS server, cutting cross-continental round trips down to regional/local latency (typically **<20–30 ms**).
- This also provides **built-in redundancy**: if one PoP fails, BGP reroutes traffic to the next-nearest healthy node automatically, without any client-side reconfiguration — directly addressing the "continuous availability during server failures" requirement.

b) TTL Optimization

- Set TTL values strategically based on record volatility:
 - **Static records** (e.g., root domain, CDN endpoints that rarely change): TTL of **3,600–86,400 seconds (1–24 hrs)** to maximize cache hits at resolver/ISP/browser level.

- **Dynamic/load-balanced records** (e.g., active failover endpoints): shorter TTL of **60–300 seconds** to allow faster failover, but not so short that it forces constant re-resolution.
- This balances **cache efficiency** (lower average lookup time) against **failover responsiveness** (ability to redirect quickly if a server goes down).

c) DNS Pre-fetching

- Enable **DNS prefetching** (<link rel="dns-prefetch">) in the web application for likely-needed domains (API endpoints, CDN assets, third-party resources) so resolution happens **in the background before the resource is actually requested**, hiding lookup latency from the user's critical path.
- Combine with **browser/OS-level DNS caching** and **prefetch hints at the application layer** so repeat visits or predictable navigation paths incur near-zero DNS delay.

d) Layered Caching Strategy

- Use a **multi-tier caching hierarchy**: local recursive resolvers at each regional PoP with their own cache, backed by a **secondary/tertiary DNS provider** for redundancy (multi-provider DNS, e.g., primary + secondary DNS vendors) so that a failure or attack on one provider doesn't take down resolution entirely.

e) Health-checked Failover Records

- Use **DNS-based load balancing with active health checks**, so that if a regional server/PoP fails, its records are automatically withdrawn or deprioritized, and traffic is routed to the next healthy Anycast node — maintaining availability without manual intervention.

1. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

Performance

- Anycast reduces propagation delay by routing to the nearest node — the single biggest contributor to hitting the <50 ms target.
- TTL optimization + prefetching drastically increase cache-hit rate, so most lookups avoid the full recursive chain entirely.
- Limitation: Anycast performance depends on **BGP routing quality**, which the company doesn't fully control — suboptimal ISP routing can occasionally send users to a farther PoP than expected ("Anycast routing anomalies").
- Very long TTLs on records that *do* need to change (e.g., during an actual failover) can cause **stale DNS caching** at ISP/client level, delaying propagation of updates for as long as the TTL, partially working against fast failover.

Scalability

- Anycast scales very well horizontally — adding new PoPs in new regions is straightforward and immediately improves latency for nearby users without re-architecting the system.
- Caching layers reduce load on authoritative servers as user base grows, since

most repeat queries are absorbed by resolvers/browsers rather than hitting origin DNS infrastructure.

- Limitation: Operating many geographically distributed PoPs increases **operational complexity and cost** (infrastructure, monitoring, BGP peering agreements, security hardening at each site).

Availability

- Anycast + multi-provider DNS gives strong resilience: a failed PoP is automatically bypassed via BGP without any client-visible downtime, and a full provider outage is mitigated by the secondary DNS vendor.
- Health-checked failover records allow automatic rerouting away from unhealthy origin servers, supporting the "continuous availability" requirement.
- Limitation: Anycast failover, while fast at the routing layer, is not instantaneous — BGP convergence can take **seconds** in worst cases, so it's not a zero-downtime guarantee, just a significant improvement over static Unicast.
- Multi-provider DNS setups add **configuration complexity and potential for inconsistency** between providers if not carefully synchronized.

3. A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

1. Examine the impact of centralized DNS architecture on application performance.

Routing all global queries to one primary DNS server causes several compounding issues:

- **Geographic latency:** Users far from the server (e.g., Asia querying a US-based server) face 150–300+ ms of pure propagation delay before resolution even starts.
- **Server overload:** During promotional spikes, millions of simultaneous queries overwhelm the server's finite CPU/network capacity, causing queuing delays.

- **Single point of failure:** If the server slows or crashes, DNS resolution — and thus the entire application — becomes unreachable globally, with no automatic failover.
- **Cascading effect:** Since DNS resolution happens before any TCP/TLS/HTTP request, this 800 ms delay is added in full to every page load, making the whole app feel slow even if backend servers are healthy.

1. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.

Anycast DNS with layered caching and redundancy:

- **Multi-region Anycast deployment:** Authoritative DNS servers at PoPs across continents, all sharing one Anycast IP. BGP automatically routes each query to the nearest healthy PoP.
- **Horizontal scaling per PoP:** Multiple server instances behind a load balancer within each region, so local spikes don't overwhelm one machine.
- **Multi-tier caching:** Recursive resolvers at each PoP cache popular records, absorbing repeat queries during traffic surges (critical when millions request the same domain simultaneously).
- **Multi-provider DNS:** A secondary independent DNS vendor as backup authoritative source, protecting against a full outage of the primary provider.
- **Health-checked routing:** Unhealthy nodes are automatically withdrawn from BGP/load-balancing, rerouting traffic without manual intervention.

1. Analyze how **TTL values**, **Anycast routing**, and **DNS caching** influence system performance and fault tolerance.

Factor	Performance impact	Fault tolerance impact
High TTL	Maximizes cache hits, reduces load on authoritative servers	Stale records may persist after a failure until TTL expires
Low TTL	Forces frequent re-resolution, more server load	Failover changes propagate quickly
Anycast routing	Routes queries to nearest node, cutting cross-continental delay and naturally load-balancing traffic	Automatic BGP-level failover to next-nearest PoP if one fails, without DNS record changes
DNS caching (multi-layer)	Absorbs repeat queries near the user, enabling scale to millions of users	Provides a temporary resilience buffer — cached answers still work briefly even if authoritative servers go down

4. A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering **flow control, Nagle's algorithm, and SYN queue limitations**.

a) Nagle's Algorithm Nagle's algorithm delays sending small packets, buffering them until either an ACK is received or enough data accumulates, in order to reduce the number of small packets on the network.

b) TCP Flow Control (Receive Window) Flow control uses the receiver's advertised window (rwnd) to prevent the sender from overwhelming the receiver's buffer. If the receiver's buffer fills up or isn't scaled properly, the window shrinks and the sender is forced to slow down or pause transmission.

c) SYN Queue Limitations New TCP connections require a three-way handshake, and incoming SYN requests wait in a SYN queue (backlog) before being accepted. If this queue is undersized, or the application doesn't accept connections fast enough, new SYNs get delayed, dropped, or retransmitted.

1. Explain how each factor affects transaction processing in high-frequency systems

a) Nagle's Algorithm: Order and acknowledgment messages in trading systems are small (often just a few hundred bytes). Nagle's algorithm holds these small packets back instead of sending them immediately, adding delay of several to tens of milliseconds per message. In a system where every millisecond matters and messages must be sent instantly, this buffering directly causes the observed latency spike, especially since order volume — and hence the number of small messages — surges at market open.

b) Flow Control: During market opening hours, order volume spikes sharply. If the receiving side (e.g., the exchange gateway) cannot drain its buffer as fast as data arrives, the advertised receive window shrinks. This forces the sender to throttle transmission, causing orders to queue up waiting for window space to open — increasing acknowledgment latency precisely during the highest-load period, matching the reported pattern.

c) SYN Queue Limitations: If the platform opens many new connections around market open (new client sessions, reconnects after brief drops, etc.), a small SYN backlog means incoming connection requests exceed the queue capacity. This causes SYN packets to be delayed or dropped, forcing

retransmissions and longer connection establishment times — adding latency before any actual order data can even be transmitted, consistent with the observed longer connection setup times and increased retransmissions.

1. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.
 - **Disable Nagle's algorithm** by setting `TCP_NODELAY` on trading connections, so small order/acknowledgment messages are sent immediately instead of being buffered.
 - **Tune flow control and buffer sizes:** Increase `SO_RCVBUF/SO_SNDBUF` and enable TCP window scaling so the receive window doesn't become a bottleneck under high message rates; ensure the application layer drains the socket buffer quickly to keep the window open.
 - **Increase SYN backlog capacity:** Tune `net.core.somaxconn` and `net.ipv4.tcp_max_syn_backlog` so connection bursts at market open are queued and accepted without drops or delays.
 - **Use persistent, pooled connections:** Maintain long-lived TCP connections between trading clients and the exchange gateway rather than repeatedly establishing new ones, avoiding handshake overhead during peak load.
 - **Enable TCP Fast Open (TFO):** Allows data to be included in the initial SYN packet, saving one round trip during connection setup.
 - **Ensure Selective Acknowledgment (SACK) is enabled:** So that only genuinely lost segments are retransmitted, rather than causing broader retransmission overhead.
 - **Consider kernel-bypass or UDP-based approaches** (e.g., DPDK, RDMA, UDP multicast for market data) for the most latency-critical paths, since TCP tuning alone has a practical latency floor that may not meet sub-millisecond requirements.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand